

```

/// <summary>
/// A recursive function to split an array into smaller sub-arrays to sort
/// </summary>
/// <param name="A">The array to sort</param>
/// <param name="first">The index of the first element</param>
/// <param name="last">The index of the last element</param>
/// <returns></returns>
static double[] MergeSort(double[] A, int first, int last)
{
    if (A.Length != 1)
    {
        int half = A.Length / 2;
        double[] Asub1 = A.Take(half).ToArray();
        double[] Asub2 = A.Skip(half).ToArray();

        double[] B = MergeSort(Asub1, 0, Asub1.Length - 1);
        double[] C = MergeSort(Asub2, 0, Asub2.Length - 1);

        return Merge(MergeSort(Asub1, 0, Asub1.Length - 1), MergeSort(Asub2, 0, Asub2.Length - 1),
Asub1.Length, Asub2.Length);
    }
    else
        return A;
}

/// <summary>
/// Merges two sorted arrays into one array that is also sorted
/// </summary>
/// <param name="B">The first array</param>
/// <param name="C">The second array</param>
/// <param name="bSize">The size of the first array</param>
/// <param name="cSize">The size of the second array</param>
/// <returns></returns>
static double[] Merge(double[] B, double[] C, int bSize, int cSize)
{
    int size = bSize + cSize;
    double[] D = new double[size];

    for (int i = 0, j = 0, k = 0; k < size; k++)
        D[k] = (i < B.Length) ? (j < C.Length) ? (B[i] < C[j]) ? B[i++] : C[j++] : B[i++] : C[j++];

    return D;
}

```